

8.5. Двоичное дерево

«Идет направо - песнь заводит,
налево – сказку говорит»
А.С. Пушкин «Лукоморье»

Упорядочение справа налево

В дереве с двумя потомками возможен еще один порядок обхода, который условно можно назвать «слева-направо»: сначала нумеруются вершины правого поддерева, затем корневая, а потом уже – левого. В принципе, алгоритмы, приведенные в 8.3 для дерева с упорядочением «сверху-вниз», можно использовать с небольшими изменениями: в рекурсивных алгоритмах меняются местами ветви, относящиеся к корневой вершине и левому поддереву. Поэтому здесь можно ограничиться лишь фрагментарными иллюстрациями

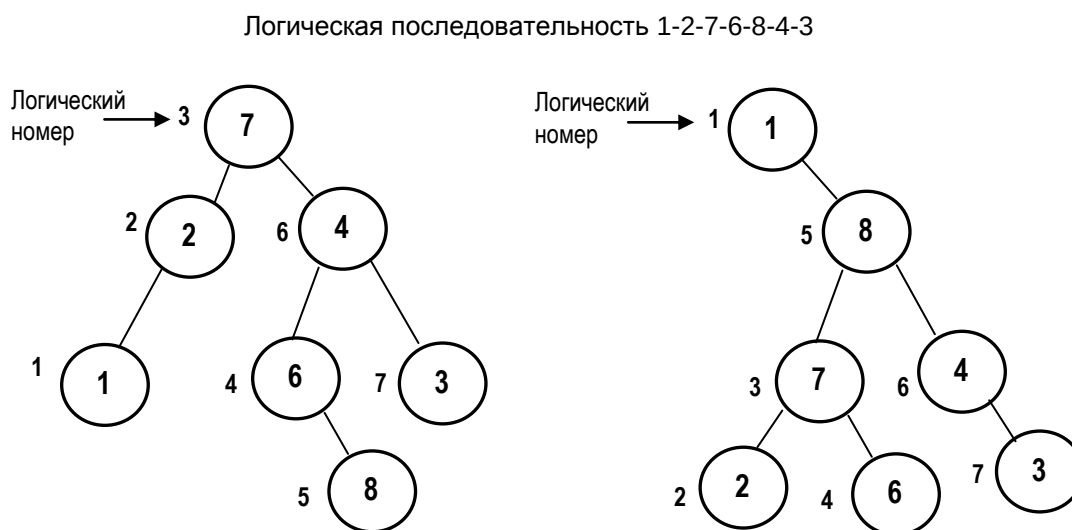
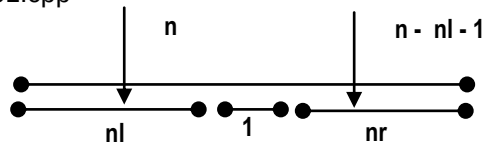


рис. 85.1. Дерево с нумерацией вершин «справа-налево»

```
//-----85-01.cpp
// Линейная СД на основе дерева с двумя потомками
// Обход дерева слева - направо (левое - текущая - правое)
void scan(tree2 *p, int level, int &ln){
    if (p==NULL) return;
    scan(p->l, level+1,ln);
    printf("l=%d n=%d cnt=%d :%s\n", level, ln, p->cnt, p->s);
    ln++;
    scan(p->r, level+1,ln); }
// Поиск вершины по логическому номеру
tree2 *get_n(tree2 *p, int n){
    if (p==NULL) return NULL;
    if (n >= p->cnt) return NULL;
    if (p->l!=NULL){
        int ll=p->l->cnt;
        if (n<ll) return get_n(p->l,n);
        n-=ll;
    }
    if (n-- ==0) return p;
    return get_n(p->r,n);
}
// Сначала – левое поддерево
// Корневая вершина
// Остаток – правом поддереве
```

При включении по логическому номеру замещение данных в текущей вершине производится с учетом изменившегося способа нумерации: если левое поддерево пустое, то новая вершина включается туда (логически – перед текущей), иначе содержимое текущей вершины сносится в правое поддерево с логическим номером 0 (логически – после текущей).

```
//-----85-01.cpp
// Включение по логическому номеру
void insert(tree2 *&p, int n, char *ss){
    if (p == NULL) { p=create(ss); return; }
    if (n >= p->cnt) return;
    p->cnt++;
    if (p->l!=NULL){
        int ll=p->l->cnt;
        if (n<ll) { insert(p->l,n,ss); return; }
        n-=ll;
    }
    if (n-- ==0) {
        if (p->l==NULL) p->l=create(ss); // Левое поддерево пустое – новая вершина там
        else { insert(p->r,0,p->s); // Иначе - вытеснить текущее в правое поддерево
            p->s=ss; } // с ЛН=0
    }
    else insert(p->r,n,ss); } // Остаток – в правое поддерево
```



```
//-----85-01.cpp
// Балансировка дерева
tree2 *balance(tree2 *pp[], int a, int b){ // Создание дерева
    if (a>b) return NULL;
    tree2 *q;
    if (a==b) { q=pp[a]; q->l=q->r=NULL; q->cnt=1; return q; }
    int m=(a+b)/2;
    q=pp[m]; // Корневая вершина – середина интервала
    q->l=balance(pp,a,m-1); // Создание поддеревьев на правом и левом
    q->r=balance(pp,m+1,b); // интервалах
    q->cnt=b-a+1;
    return q;
}

// Обход дерева с заполнением массива указателей на вершины
void set(tree2 *pp[], tree2 *p, int &ln){
    if (p==NULL) return; // ln - порядковый (логический) номер
    set(pp,p->l,ln); // рекурсивно выполнить для левого поддерева
    pp[ln++]=p; // запомнить указатель на текущую вершину
    set(pp,p->r,ln); // рекурсивно выполнить для правого поддерева
```

```
void balance(tree2 *&ph){
    int sz=ph->cnt;
    tree2 **pp=new tree2*[sz];
    int ln=0; set(pp,ph,ln);
    ph=balance(pp,0,sz-1); }
```

Двоичное дерево

Нумерация «слева-направо» дает нам основу для создания другого вида деревьев, включение в которые производится не по логическому номеру, а с учетом естественного упорядочения значений. Такое дерево называется **деревом двоичного поиска** или просто **двоичным деревом**. Основное свойство, рекурсивно соблюдаемое для всех поддеревьев, звучит так: в левом поддереве каждой вершины содержатся значения, меньшие чем в корневой, а в правом – большие. Отсюда следуют все прочие свойства такого дерева:

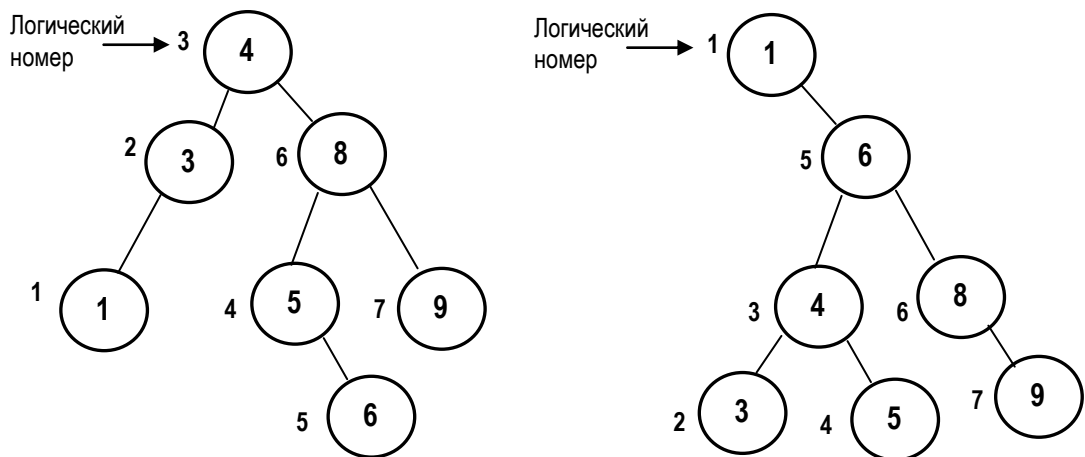


рис. 85.2. Двоичное дерево

- обход двоичного дерева «слева-направо» соответствует порядку возрастания значений, хранящихся в нем;
- поиск заданного значения в двоичном дереве использует линейно рекурсивный (или циклический) алгоритм. В каждой вершине производится сравнение с искомым значением (ключом). Если содержимое вершины совпадает с искомым, то вершина найдена. Если искомое значение меньше, то оно ищется в левом поддереве, иначе – в правом. Данный алгоритм очень сильно напоминает нам алгоритм **двоичного поиска в массиве** (см.4.6). Там одно сравнение выбирает половину интервала, здесь – поддерево;
- включение нового значения в двоичного дерева также базируется на ветвлении: при сравнении включаемого значения со значением в текущей вершине выбирается правая или левая ветвь до тех пор, пока не встретится свободная.

```
//-----85-02.cpp
// Двоичное дерево
// Обход дерева слева - направо (левое - текущая - правое)
struct btree{
    int cnt;           // Количество вершин в дереве
    char *s;
    btree *l,*r;       // Левый и правый потомки
};

// Поиск вершины с заданным значением - рекурсия
btree *search(btree *p, char *key){
    if (p==NULL) return NULL;
    int k=strcmp(key,p->s);
    if (k==0) return p;
    if (k<0) return search(p->l,key);
    return search(p->r,key);
}

// Поиск вершины с заданным значением - цикл
btree *search1(btree *p, char *key){
    while(p!=NULL){
        int k=strcmp(key,p->s);
        if (k==0) return p;
        if (k<0) p=p->l;
    }
}
```

```

        else p=p->r;
    }
    return NULL; }

```

// Включение двоичное дерево (с сохранением порядка)

```

void insert(btree *&p, char *ss){
    if (p == NULL) { p=create(ss); return; }
    p->cnt++;
    if (strcmp(ss,p->s)<0)
        insert(p->l,ss);
    else
        insert(p->r,ss);
}

```

Все остальные процедуры (извлечение и удаление по логическому номеру, обход, балансировка) идентичны предыдущему варианту – дереву с обходом «справа-налево». Но по своей природе двоичное дерево соответствует идее **структурной упорядоченности данных**. Эта упорядоченность присутствует с самого определения дерева. Поэтому простое включение последовательности значений в двоичное дерево уже является их сортировкой: для этого нужно просто сделать обход полученного дерева. Что же касается трудоемкости такого процесса, то она зависит от вида полученного дерева: если оно сбалансировано, то получаем $T=N\log_2 N$, если вырождается в линейный список, то $T=N^2/2$.

Вид полученного двоичного дерева в каждом случае будет различен. Это зависит от последовательности значений включаемых данных. Если в них будет цепочка одинаковых или возрастающих значений, то они создадут линейную цепочку правых потомков, которая уменьшает сбалансированность. Если данные исходно упорядочены, то при включении их в двоичное дерево последнее вырождается в правосторонний список.



Короче, хорошая идея на идеальных данных дает отвратительный результат (плюс на плюс дает минус).

Двоичное дерево в массиве

Аналогично пирамиде, построенной в массиве для дерева, упорядоченного в глубину (см.8.4), двоичное дерево также можно спроецировать на линейный массив по принципу: корневая вершина с индексом n имеет левого и правого потомков с индексами $2n$ и $2n+1$. При использовании динамического массива в процедуре включения данных в дерево необходимо перераспределять память при выходе алгоритма за его пределы.

В данном примере для хранения упорядоченной последовательности строк используется массив указателей. Отсутствующие вершины обозначаются NULL-указателями. Аналогично, выход алгоритма за пределы массива обозначает отсутствие вершины.

```

//-----85-03.cpp
// Двоичное дерево в массиве (массив указателей на строки)
struct btree{
    char **p;           // дерево в массиве указателей на строки
    int sz;              // текущая размерность массива
}
//-----
void init(){ sz=10; p=new char*[sz]; // NULL-указатель для свободной вершины
    for (int i=0;i<sz;i++) p[i]=NULL;
}
//-- Число вершин в поддереве
int size(int n){
    if (n>=sz || p[n]==NULL) return 0;
    return 1+size(2*n)+size(2*n+1); }

```

```

//-- Обход дерева
void scan(int n, int level, int &ln){
    if (n>=sz || p[n]==NULL) return;
    scan(2*n,level+1,ln);
    printf("l=%d n=%d :%s\n", level, ln, p[n]);
    ln++;
    scan(2*n+1,level+1,ln);}

//-- Поиск вершины по логическому номеру
char *get_n(int m, int n){
    if (m>=sz || m>=size(n)) return NULL;
    int ll=size(2*n); // число вершин в левом поддереве
    if (m<ll) return get_n(m,2*n); // ЛН в левом поддереве
    m-=ll; // отбросить вершины левого поддерева
    if (m-- ==0) return p[n]; // ЛН – номер текущей вершины
    return get_n(m,2*n+1); // в правое поддерево с остатком ЛН
}

// Включение с сохранением порядка
void insert(int n, char *ss){
    if (n>=sz){ // увеличение размерности при выходе
        sz*=2; // за пределы массива
        p=(char**)realloc(p,sz*sizeof(char*));
        for (int i=sz/2;i<sz;i++) p[i]=NULL; // с обнулением новой части
    }
    if (p[n] == NULL) { p[n]=ss; return; } // свободная вершина - включение
    if (strcmp(ss,p[n])<0) // выбор левого или правого поддерева
        insert(2*n, ss); // в зависимости от результата сравнения
    else
        insert(2*n+1, ss);}

```

Для балансировки двоичного дерева используется обход с сохранением упорядоченной последовательности в линейном массиве (массиве указателей). Затем массив рекурсивно делится пополам, а значение из середины интервала включается в новое дерево, которое получается сбалансированным.

```

//-----85-03.cpp
//-- обход дерева с сохранением строк в линейном массиве
void set(char *pp[], int n, int &ln){
    if (n>=sz || p[n]==NULL) return;
    set(pp,2*n,ln);
    pp[ln++]=p[n];
    set(pp,2*n+1,ln);}

// Построение сбалансированного дерева
void balance(char *p[], int a, int b){
    if (a>b) return;
    int m=(a+b)/2; // взять строку из середины интервала
    insert(1,p[m]); // и включить в двоичное дерево
    balance(p,a,m-1); // рекурсивно выполнить для левой и
    balance(p,m+1,b); // правой частей
}

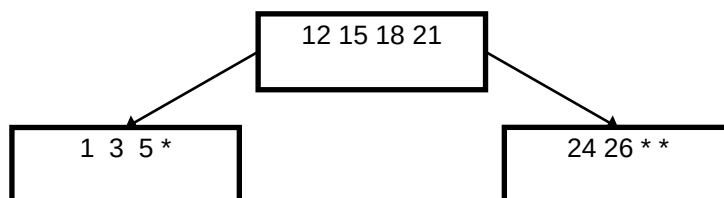
// Балансировка дерева
void balance(){
    int sz1=size(1),ln=0;
    char **pp=new char*[sz1];
    set(pp,1,ln);
    delete p;
    init();
    balance(pp,0,sz1-1);
}

```

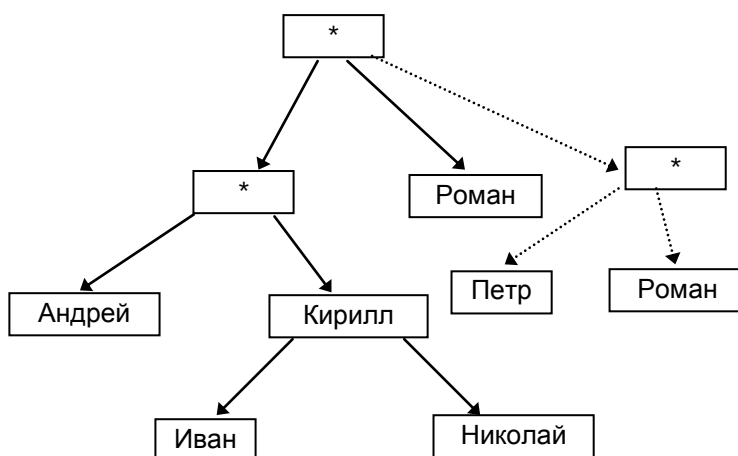
Лабораторный практикум

Программа должна содержать функцию обхода дерева с выводом его содержимого, функцию добавления вершины дерева (ввод), а также указанную в варианте функцию.

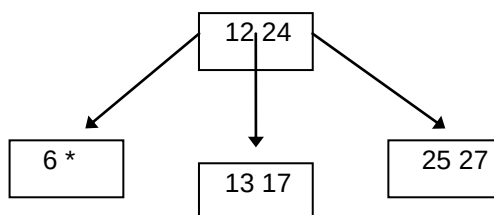
1. Вершина двоичного дерева содержит массив целых и два указателя на правое и левое поддереву. Массив целых в каждом элементе упорядочен, дерево в целом также упорядочено. Функция включает в дерево целую переменную с сохранением упорядоченности.



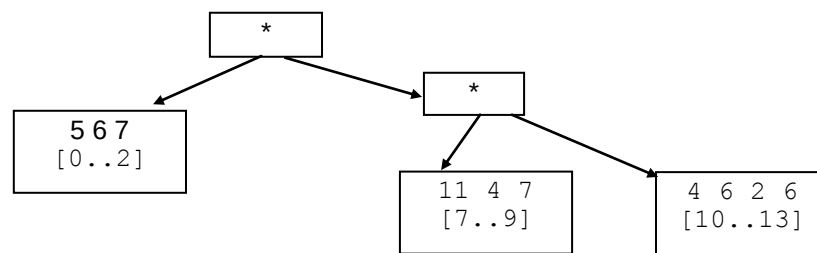
2. Элемент дерева содержит либо данные (строка текста), либо указатели на правое и левое поддеревья. Строки в дереве упорядочены в порядке обхода концевых вершин. Написать функцию включения новой строки. (Обратить внимание на то, что элемент с указателями не содержит данных, и при включении новой вершины вершину с данными следует заменить на вершину с указателями).



3. Вершина дерева содержит два целых числа и три указателя на поддеревья. Данные в дереве упорядочены. Написать функцию включения нового значения в дерево с сохранением упорядоченности.



4. Вершина дерева содержит указатель на строку и N указателей на потомков. Функция помещает строки в дерево так, что строки с меньшей длиной располагаются ближе к корню. Если новая строка «проходит» через вершину, в которой находится более длинная строка, то новая занимает место старой, а алгоритм включения продолжается для старой строки. Функция включения выбирает потомка минимальным количеством вершин в поддереве.
5. Вершина дерева содержит либо 4 целых значения, либо 2 указателя на потомков, причем концевые вершины содержат данные, а промежуточные - указатели на потомков. Естественная нумерация значений производится при обходе концевых вершин слева направо. Разработать функции получения и включения значения в дерево по логическому номеру.



6. Вершина дерева содержит N целых значений и два указателя на потомков. Запись значений производится таким образом, что меньшие значения оказываются ближе к корню дерева (то есть все значения в поддеревьях больше самого большого значения у предка). Разработать функции включения и поиска данных в таком дереве. Если новое значение «проходит» через вершину, в которой находится большее, то оно замещает большее значение, а для последнего алгоритм включения. Функция включения выбирает потомка максимальным значением в поддереве.
7. Выражение, содержащее целые константы, арифметические операции и скобки, может быть представлено в виде двоичного дерева. Концевая вершина дерева должна содержать значение константы. Промежуточная - код операции и указатели на правый и левый операнды - вершины дерева. Функция получает строку, содержащую выражение, и строит по ней дерево. Другая функция производит вычисления по полученному дереву.
8. Вершина дерева содержит указатель на строку и динамический массив указателей на потомков. Размерность динамического массива в корневой вершине - N, на каждом следующем уровне - в 2 раза больше. Функция при включении строки создает вершину, наиболее близкую к корню.
9. Вершина дерева содержит указатель на строку и список указателей на потомков. Размерность списка в корневой вершине - N, на каждом следующем уровне - в 2 раза больше. Функция при включении строки создает вершину, наиболее близкую к корню.
10. Вершина дерева содержит динамический массив целых значений и два указателя на потомков. Значения в дереве не упорядочены. Размерность динамического массива в корневой вершине - N, на каждом следующем уровне - в 2 раза больше. Функция включает новое значение в свободное место в массиве ближайшей к корню вершины.
11. Вершина дерева содержит массив целых и два указателя на правое и левое поддерево. Значения в дереве не упорядочены. Естественная нумерация значений

производится путем обхода дерева по принципу «левое поддерево - вершина - правое поддерево». Разработать функции включения и получения значения элемента по заданному логическому номеру.

12. Код Хаффмана, учитывающий частоты появления символов, строится следующим образом. Для каждого символа подсчитывается частота его появления и создается вершина двоичного дерева. Затем из множества вершин выбираются две с минимальными частотами появления и создается новая - с суммарной частотой, к которой выбранные подключаются как правое и левое поддерево. Созданная вершина включается в исходное множество, а выбранные - удаляются. Затем процесс повторяется до тех пор, пока не останется единственная вершина. Код каждого символа - это последовательность движения к его вершине от корня (левое поддерево - 0, правое - 1). Функция строит код Хаффмана для символов заданной строки.

Вопросы без ответов

Определить по тесту способ представления дерева (массив с индексами потомков, массив с вычисляемыми индексами потомков, ветвящийся список, вершина с массивом указателей на потомков). Определить дерево в виде статической структуры данных (инициализированные переменные) и написать для нее вызов функции, заданной в тесте. Содержательно сформулировать результат ее выполнения и обосновать полученное при вызове функции значение.

```
//-----85-xx.cpp
struct ltree{
    int val;
    ltree *child,*next; };

struct tree{
    int val;
    tree *ch[4]; };

struct tree1{
    int val;
    int n;
    tree1 *ch[10]; };

struct btree{
    int val;
    btree *l,*r; };

//-----1
int F1(ltree *p){
    int n=1;
    for (ltree *q=p->child; q!=NULL; q=q->next)
        n+=F1(q);
    return n; }

//-----2
int F2(ltree *p){
    int n=p->val;
    for (ltree *q=p->child; q!=NULL; q=q->next)
        n+=F2(q);
    return n; }

//-----3
int F3(ltree *p){
    int n=p->val;
    for (ltree *q=p->child; q!=NULL; q=q->next)
        { int vv=F3(q); if (vv > n) n=vv; }
```



```

        return n; }
//-----4
int F4(ltree *p){
    int n=0;
    for (ltree *q=p->child; q!=NULL; q=q->next)
        { int vv=F4(q); if (vv > n) n=vv; }
    return n+1; }
//-----5
void F5(ltree *p, int l, int &m){
    if (l > m) m=l;
    for (ltree *q=p->child; q!=NULL; q=q->next)
        F5(q,l+1,m); }
//-----6
int F6(ltree *p, int l){
    int n=l;
    for (ltree *q=p->child; q!=NULL; q=q->next)
        n+=F6(q,l+1);
    return n; }
double main6(ltree *p){
    return ((double)F6(p,1))/F1(p);
}
//-----7
int F7(ltree *p){
    if (p->val > 6) return p->val;
    for (ltree *q=p->child; q!=NULL; q=q->next)
        { int vv=F7(q); if (vv!=-1) return vv; }
    return -1; }
//-----8
int F8(btree *p) {
    if (p==NULL) return 0;
    return (1 + F8(p->r) + F8(p->l)); }
//-----9
int F9(btree *p) {
    if (p==NULL) return 0;
    int nr=F9(p->r);
    int nl=F9(p->l);
    return 1 + (nr>nl ? nr : nl); }
//-----10
int F10(btree *p) {
    if (p==NULL) return 0;
    int m,n=p->val;
    if ((m=F10(p->r))>n) n=m;
    if ((m=F10(p->l))>n) n=m;
    return n; }
//-----11
int F11(tree1 *p){
    int s=1;
    for (int i=0; i < p->n; i++)
        s+=F11(p->ch[i]);
    return s; }
//-----12
int F12(tree1 *p){
    int s=p->val;
    for (int i=0; i < p->n; i++)
        s+=F12(p->ch[i]);
    return s; }
//-----13
int F13(tree1 *p){
    int s=p->val;
    for (int i=0; i < p->n; i++)
        { int vv=F13(p->ch[i]); if (vv > s) s=vv; }
    return s; }
//-----14
int F14(tree1 *p){

```

```

        int s=0;
        for (int i=0;i < p->n; i++)
            { int vv=F14(p->ch[i]);    if (vv > s) s=vv; }
        return s+1; }
//-----15
void F15(tree1 *p, int l, int &m){
    if (l > m) m=l;
    for (int i=0;i < p->n; i++)
        F15(p->ch[i],l+1,m); }
//-----16
int F16(tree1 *p, int l){
    int s=l;
    for (int i=0;i < p->n; i++)
        s+=F16(p->ch[i],l+1);
    return s; }
double main16(tree1 *p){
    return ((double)F16(p,1))/F11(p); }
//-----17
int F17(tree1 *p){
    if (p->val > 6) return p->val;
    for (int i=0;i < p->n; i++)
        { int vv=F17(p->ch[i]); if (vv!=-1) return vv; }
    return -1; }
//-----21
int F21(tree *p){
    if (p==NULL) return 0;
    int s=1;
    for (int i=0; i<4; i++)
        s+=F21(p->ch[i]);
    return s; }
//-----22
int F22(tree *p){
    if (p==NULL) return 0;
    int s=p->val;
    for (int i=0; i<4; i++)
        s+=F22(p->ch[i]);
    return s; }
//-----23
int F23(tree *p){
    if (p==NULL) return 0;
    int s=p->val;
    for (int i=0; i<4; i++)
        { int vv=F23(p->ch[i]);    if (vv > s) s=vv; }
    return s; }
//-----24
int F24(tree *p){
    if (p==NULL) return 0;
    int s=0;
    for (int i=0; i<4; i++)
        { int vv=F24(p->ch[i]);    if (vv > s) s=vv; }
    return s+1; }
//-----25
void F25(tree *p, int l, int &m){
    if (p==NULL) return;
    if (l > m) m=l;
    for (int i=0; i<4; i++)
        F25(p->ch[i],l+1,m); }
//-----26
int F26(tree *p, int l){
    if (p==NULL) return 0;
    int s=l;
    for (int i=0; i<4; i++)
        s+=F26(p->ch[i],l+1);
    return s; }

```

```

double main26(tree *p){
    return ((double)F26(p,1))/F21(p); }
//-----27
int F27(tree *p){
    if (p->val > 6) return p->val;
    for (int i=0; i<4; i++)
        { int vv=F27(p->ch[i]); if (vv!=-1) return vv; }
    return -1; }
//-----30
int F30(int A[],int sz, int n){
    if (n>=sz || A[n]==-1) return 0;
    return 1+F30(A,sz,2*n)+F30(A,sz,2*n+1); }
//-----31
int F31(int A[],int sz, int n){
    if (n>=sz || A[n]==-1) return 0;
    return A[n]+F31(A,sz,2*n)+F31(A,sz,2*n+1); }
//-----32
int F32(int A[],int sz, int n){
    if (n>=sz || A[n]==-1) return 0;
    int xx,vv=A[n];
    if ((xx=F32(A,sz,2*n))>vv) vv=xx;
    if ((xx=F32(A,sz,2*n+1))>vv) vv=xx;
    return vv; }
//-----33
int F33(int A[],int sz, int n){
    if (n>=sz || A[n]==-1) return 0;
    int x1=F33(A,sz,2*n);
    int x2=F33(A,sz,2*n+1);
    return x1>x2 ? x1+1 : x2+1; }

```